TRABAJO PRÁCTICO · UNIDAD 7: NoSQL Y NUEVAS FORMAS DE ALMACENAMIENTO

Modelado Documental, Migración y Comparación de Consultas: Food Store (PostgreSQL vs. MongoDB)

Institución: Tecnicatura Universitaria en Programación | **Cátedra:** Bases de Datos II
Alumno: Maximiliano Nardez | **Fecha:** Noviembre 2026 | **Repositorio:** https://github.com/TekkNo0/TPU7
Ambiente Técnico: PostgreSQL 17 (Relacional, puerto 5432) | MongoDB 9.0.4 / mongosh 2.14.1 (Documental, puerto 27017)

1. Alcance y Objetivos de la Práctica

El presente trabajo implementa la representación en **MongoDB** de un subconjunto transaccional crítico del proyecto **Food Store**: la gestión de pedidos y sus líneas de detalle. El propósito es contrastar la arquitectura relacional normalizada frente al paradigma documental orientado a documentos agregados (patrón *Embedding*), evaluando el impacto sobre la consistencia financiera, la localidad de referencia en lecturas y la semántica de consulta, contrastando rigurosamente las recomendaciones provistas por IA.

2. Parte 1 · Modelo Documental y Correspondencia (Consignas 1, 2 y 3)

2.1 Identificación del Submodelo Relacional en PostgreSQL:

El subconjunto seleccionado involucra cuatro tablas de PostgreSQL vinculadas mediante claves primarias (PK) y foráneas (FK):
• **pedido (Cabecera):** Clave primaria id (Identity). Clave foránea usuario_id → usuario(id). Atributos: fecha, estado (enum), forma_pago, total, metadata (jsonb), eliminado.
• **detalle_pedido (Líneas):** Clave primaria id. Claves foráneas: pedido_id → pedido(id), producto_id → producto(id). Atributos: cantidad (CHECK > 0), precio_unitario (CHECK ≥ 0). Restricción UNIQUE(pedido_id, producto_id).
• **usuario (Cliente):** Clave primaria id. Atributos esenciales: nombre, apellido, email (UNIQUE), telefono.
• **producto (Catálogo):** Clave primaria id. Atributos: nombre, precio, stock, categoria_id.

2.2 Esquema Documental Propuesto en MongoDB (Colección pedidos):

Se adoptó un modelo desnormalizado incrustado (*Embedding*). Se conservaron los identificadores numéricos de origen: el campo _id preserva biyectivamente el id de pedido de PostgreSQL, permitiendo trazabilidad exacta. El documento agrupa los datos del comprador en un objeto embebido cliente y las líneas en un arreglo items.

```
{
  "_id": 1,                     // Conserva pedido.id de PostgreSQL (trazabilidad exacta)
  "pedido_id": 1,
  "fecha": "2025-01-01T22:00:00-03:00",
  "estado": "CONFIRMADO",
  "forma_pago": "TARJETA",
  "total": 5464.71,             // Coincide exactamente con la sumatoria de las líneas
  "cliente": {                  // Objeto embebido (Snapshot de datos esenciales del cliente)
    "usuario_id": 2, "nombre": "Usuario2", "apellido": "Apellido2",
    "email": "usuario2@mail.com", "telefono": "541100000014"
  },
  "items": [                    // Arreglo de líneas de pedido (Desnormalización controlada)
    {
      "detalle_id": 1, "producto_id": 7, "producto_nombre": "Producto Snacks 7 507ml",
      "cantidad": 3, "precio_unitario": 759.49, "subtotal": 2278.47
    },
    {
      "detalle_id": 2, "producto_id": 8, "producto_nombre": "Producto Salsas 8 508ml",
      "cantidad": 4, "precio_unitario": 796.56, "subtotal": 3186.24
    }
  ],
  "cantidad_items": 2
}
```

2.3 Reglas de Integridad y Justificación de Embedding (Consigna 3):

• **Regla 1: Consistencia Financiera e Inmutabilidad de Precios Históricos:** En transacciones comerciales de Food Store, el precio y subtotal pactados en el momento de la compra son definitivos. Se debe garantizar que la sumatoria de las líneas embebidas coincida con el campo total (Δ = $0.00). Si el precio del producto cambia en el catálogo futuro, el precio congelado en el documento del pedido **no debe sufrir alteraciones**.
• **Regla 2: Integridad de Entidades y No Nulidad:** Todo pedido debe asociarse a un cliente existente y contener al menos una línea con cantidad positiva (CHECK cantidad > 0). Al incrustarse cliente e ítems en la misma entidad, se garantiza atomicidad estructural total.
• **Justificación técnica del patrón Embedding (Doc Oficial MongoDB):**
1) *Localidad de datos y rendimiento I/O:* La consulta típica de Food Store (ver pedido, despacho o factura) recupera el documento completo en una **única operación secuencial de disco (un solo I/O)**, eliminando JOINs relacionales y etapas $lookup costosas.
2) *Atomicidad a nivel de documento:* Las escrituras sobre un único documento son ACID de forma nativa en MongoDB, permitiendo crear o transicionar el pedido y todas sus líneas de modo indivisible sin transacciones distribuidas.

3. Parte 2 · Migración y Consultas en MongoDB (Consignas 2 y 4)

Se migraron 10 pedidos reales desde PostgreSQL (IDs 1 al 10) con 30 líneas de detalle mediante el script consultas_mongodb.js. A continuación se detallan las 4 consultas exigidas y sus evidencias de ejecución real obtenidas mediante mongosh:

Consigna 4.a: Mostrar todos los pedidos, incluyendo su identificador y estado

Instrucción: db.pedidos.find({}, { _id: 1, estado: 1 });

```
Pedido ID: 1 | Estado: CONFIRMADO       Pedido ID: 6 | Estado: ENTREGADO
Pedido ID: 2 | Estado: CONFIRMADO       Pedido ID: 7 | Estado: ENTREGADO
Pedido ID: 3 | Estado: CONFIRMADO       Pedido ID: 8 | Estado: ENTREGADO
Pedido ID: 4 | Estado: CONFIRMADO       Pedido ID: 9 | Estado: CANCELADO
Pedido ID: 5 | Estado: CONFIRMADO       Pedido ID: 10 | Estado: PENDIENTE
```

Consigna 4.b: Filtrar pedidos por un estado que exista en los datos migrados ('CONFIRMADO')

Instrucción: db.pedidos.find({ estado: "CONFIRMADO" }, { _id: 1, estado: 1, total: 1, forma_pago: 1, "cliente.nombre": 1, "cliente.apellido": 1 });

```
Pedido ID: 1 | Estado: CONFIRMADO | Total: $5464.71  | Pago: TARJETA       | Cliente: Usuario2 Apellido2
Pedido ID: 2 | Estado: CONFIRMADO | Total: $14802.79 | Pago: TRANSFERENCIA | Cliente: Usuario3 Apellido3
Pedido ID: 3 | Estado: CONFIRMADO | Total: $30990.02 | Pago: MERCADO_PAGO  | Cliente: Usuario4 Apellido4
Pedido ID: 4 | Estado: CONFIRMADO | Total: $55917.90 | Pago: TARJETA       | Cliente: Usuario5 Apellido5
Pedido ID: 5 | Estado: CONFIRMADO | Total: $10499.23 | Pago: EFECTIVO      | Cliente: Usuario6 Apellido6
[Resultado: 5 pedidos recuperados de los 10 migrados]
```

Consigna 4.c: Buscar pedidos que contengan un producto determinado dentro del arreglo de líneas

Se seleccionó el producto_id = 8 ('Producto Salsas 8 508ml') incrustado dentro del arreglo items:

Instrucción: db.pedidos.find({ "items.producto_id": 8 }, { _id: 1, estado: 1, total: 1, "cliente.nombre": 1, items: 1 });

```
Pedido ID: 1 | Total: $5464.71 | Estado: CONFIRMADO | Cliente: Usuario2
   -> Item: Producto Snacks 7 507ml (Prod ID: 7) | Cant: 3 | P.Unit: $759.49 | Subtotal: $2278.47
   -> Item: Producto Salsas 8 508ml (Prod ID: 8) | Cant: 4 | P.Unit: $796.56 | Subtotal: $3186.24
[Resultado: 1 pedido recuperado (ID: 1) que contiene exactamente la línea de producto especificada]
```

Consigna 4.d: Buscar pedidos cuyo total sea superior a un importe ($15.000) y justificación

Justificación del umbral de $15.000: En el análisis de ventas de Food Store, los consumos individuales y almuerzos rápidos promedian entre $3.000 y $12.000. El umbral de $15.000 es la cota operativa que identifica pedidos de grupos familiares o corporativos de alto valor.

Instrucción: db.pedidos.find({ total: { $gt: 15000 } }, { _id: 1, total: 1, estado: 1, "cliente.nombre": 1, "cliente.apellido": 1 }).sort({ total: -1 });

```
Pedido ID: 4 | Total: $55917.90 | Estado: CONFIRMADO | Cliente: Usuario5 Apellido5
Pedido ID: 3 | Total: $30990.02 | Estado: CONFIRMADO | Cliente: Usuario4 Apellido4
Pedido ID: 9 | Total: $20072.15 | Estado: CANCELADO  | Cliente: Usuario10 Apellido10
Pedido ID: 7 | Total: $17836.64 | Estado: ENTREGADO  | Cliente: Usuario8 Apellido8
Pedido ID: 8 | Total: $17392.02 | Estado: ENTREGADO  | Cliente: Usuario9 Apellido9
[Resultado: 5 pedidos recuperados cuyo importe total supera estrictamente los $15.000]
```

Control de Integridad Financiera en MongoDB (Agregación de Subtotales vs Cabecera):

```
db.pedidos.aggregate([{ $unwind: '$items' }, { $group: { _id: '$_id', cabecera: { $first: '$total' }, suma: { $sum: '$items.subtotal' } } }])
-> Pedidos 1 al 10 verificados: Diferencia cabecera vs sumatoria de ítems = $0.00 exactos en todos los documentos.
```

4. Consultas Equivalentes en PostgreSQL y Verificación Cruzada (Consigna 5)

Se ejecutaron las consultas relacionales homólogas en PostgreSQL 17 mediante psql sobre la base foodstore. A diferencia de MongoDB donde la información está incrustada, en PostgreSQL es necesario ejecutar sentencias JOIN entre pedido, usuario, detalle_pedido y producto.

```
-- Equivalente Apartado b (Estado 'CONFIRMADO' en subconjunto migrado):
SELECT p.id, p.estado, p.total, p.forma_pago, u.nombre, u.apellido
FROM pedido p JOIN usuario u ON p.usuario_id = u.id
WHERE p.estado = 'CONFIRMADO' AND p.id BETWEEN 1 AND 10 ORDER BY p.id;
--> Retorna 5 filas: IDs [1, 2, 3, 4, 5]
```

```
-- Equivalente Apartado c (Contiene producto_id = 8 en subconjunto migrado):
SELECT DISTINCT p.id, p.estado, p.total, u.nombre, dp.producto_id, pr.nombre AS prod_nombre
FROM pedido p JOIN usuario u ON p.usuario_id = u.id
JOIN detalle_pedido dp ON p.id = dp.pedido_id JOIN producto pr ON dp.producto_id = pr.id
WHERE dp.producto_id = 8 AND p.id BETWEEN 1 AND 10;
--> Retorna 1 fila: ID [1] | Cantidad: 4 | Precio: $796.56 | Subtotal: $3186.24
```

Tabla Comparativa de Verificación Exigida por la Cátedra:

Consulta Evaluada	Cantidad en PostgreSQL (Subconjunto IDs 1-10)	Cantidad en MongoDB (Colección pedidos)	¿Coinciden los IDs?	Identificadores de Pedidos Recuperados
Estado seleccionado ('CONFIRMADO')	**5**	**5**	**SÍ**	[1, 2, 3, 4, 5]
Producto seleccionado (producto_id = 8)	**1**	**1**	**SÍ**	[1]
Total superior a umbral (total > 15000)	**5**	**5**	**SÍ**	[3, 4, 7, 8, 9]

Explicación de Diferencias Detectadas y Resolución de Alcance:

1. **Coincidencia en el subconjunto migrado:** Cuando las consultas en PostgreSQL se acotan estrictamente al rango migrado (id BETWEEN 1 AND 10), la correspondencia con MongoDB es **absoluta y biyectiva (100%)** tanto en cardinalidad como en identificadores exactos.
2. **Divergencia de alcance global explicada:** Si en PostgreSQL se ejecuta la consulta sin cláusula de delimitación, el motor devuelve:
   - **40.000 pedidos** para estado CONFIRMADO (la base PostgreSQL contiene 80.000 pedidos en producción).
   - **20 pedidos** para el producto 8 a lo largo de los 80.000 pedidos.

Esta disparidad no representa una inconsistencia técnica de datos ni un error en el modelo documental, sino una consecuencia directa del alcance del laboratorio: en MongoDB se migró un subconjunto representativo de 10 pedidos con sus líneas para estudiar el comportamiento NoSQL sin replicar la totalidad de los datos relacionales.

Verificación de Coherencia Financiera en PostgreSQL 17:

Se ejecutó una consulta de agregación agrupando por pedido_id para contrastar la cabecera contra la sumatoria de líneas:
SELECT p.id, p.total, SUM(dp.cantidad * dp.precio_unitario) as suma, ROUND(p.total - SUM(dp.cantidad * dp.precio_unitario), 2) as dif FROM pedido p JOIN detalle_pedido dp ON p.id = dp.pedido_id WHERE p.id ≤ 10 GROUP BY p.id;
Resultado: Los 10 pedidos arrojaron una diferencia exacta de **$0.00**, confirmando que la integridad financiera fue preservada con éxito tanto en PostgreSQL como en la colección de MongoDB.

5. Parte 3 · Evaluación de la Tecnología e Inteligencia Artificial (Consignas 6 y 7)

5.1 Consulta Técnica Formulada a la Herramienta de IA (Prompt):

«En el sistema Food Store tenemos actualmente un modelo relacional en PostgreSQL donde los pedidos se dividen en tablas normalizadas: `pedido` (cabecera con total, fecha, estado), `detalle_pedido` (líneas con cantidad y precio unitario), `usuario` (cliente) y `producto`. El patrón de acceso principal de la aplicación operativa es la lectura frecuente de pedidos completos (para checkout, visualización en app móvil, despacho en cocina y generación de comprobantes fiscales). Además, es crítico garantizar la coherencia estricta entre la sumatoria de las líneas y el total, y asegurar que los precios históricos no se alteren si el catálogo de productos se actualiza. ¿Conviene migrar este subconjunto de pedidos a MongoDB mediante un modelo documental embebido o es preferible mantenerlo en PostgreSQL? Compara ambos motores en cuanto a almacenamiento, consultas de pedidos completos, integridad financiera y escalabilidad.»

Síntesis de la Respuesta de la IA: La IA argumentó que MongoDB optimiza drásticamente las lecturas al suprimir los JOINs relacionales, pero afirmó taxativamente que *'MongoDB no soporta transacciones ACID ni integridad referencial para pedidos'*, recomendando PostgreSQL como la única opción viable si se requiere consistencia financiera, o bien migrar completamente a MongoDB si se prioriza la latencia.

5.2 Contraste Riguroso de Dos Afirmaciones de la IA con Documentación Oficial y Laboratorio:

Afirmación 1 de la IA: *«MongoDB no soporta transacciones ACID ni integridad referencial para pedidos, por lo que cualquier fallo durante la confirmación del pedido dejará datos inconsistentes a menos que se use PostgreSQL.»*

Contraste Técnico con Documentación Oficial de MongoDB: FALSO / ANACRÓNICO.
1. **Atomicidad nativa a nivel de documento:** Según la documentación oficial (*Data Modeling - Embedding* y *Atomicity*), en MongoDB **toda operación de escritura sobre un único documento es atómica y transaccional (ACID) por diseño**. Al aplicar el patrón incrustado (embebido de cliente e ítems en el pedido), la creación completa de la orden ocurre de forma 100% atómica sin requerir bloqueos distribuidos ni transacciones multitabla pesadas.
2. **Transacciones multi-documento:** Desde **MongoDB 4.0** (y distribuido en clústeres desde 4.2), MongoDB implementa formalmente **Multi-Document ACID Transactions** con semántica session.startTransaction() y commitTransaction(), permitiendo por ejemplo coordinar la inserción del pedido y el descuento de stock atómico entre distintas colecciones con niveles de aislamiento snapshot.
3. **Validación de Esquema:** MongoDB soporta $jsonSchema en la colección, permitiendo forzar tipos de datos y rangos numéricos estrictos de forma equivalente a las restricciones CHECK relacionales.

Afirmación 2 de la IA: *«En PostgreSQL, recuperar un pedido completo siempre requiere múltiples JOINs lentos que degradan severamente el rendimiento en comparación con MongoDB, que siempre lee secuencialmente en un único I/O.»*

Contraste Técnico con Pruebas de Laboratorio y Optimizador de PostgreSQL: SIMPLIFICACIÓN SESGADA.
1. **Eficiencia del Index Scan en PostgreSQL 17:** En consultas puntuales filtradas por PK (WHERE p.id = $1), PostgreSQL realiza un Index Scan sobre la clave primaria de pedido y otro sobre el índice B-tree de detalle_pedido(pedido_id). En un servidor en producción con páginas en memoria caché (shared_buffers), la consulta se resuelve en menos de **0.5 a 1.2 milisegundos**, haciendo imperceptible la diferencia frente a MongoDB para lecturas individuales.
2. **Soporte JSON Nativo en PostgreSQL:** PostgreSQL permite generar el objeto jerárquico completo en el propio motor relacional mediante jsonb_build_object() y jsonb_agg(), enviando el documento JSON compilado directamente al cliente en un único viaje de red.
3. **Costo oculto en WiredTiger (MongoDB):** Si un pedido incrustado sufre mutaciones frecuentes (cambios sucesivos de estado o líneas adicionales), WiredTiger debe reescribir el documento BSON íntegro, provocando amplificación de escritura (*write amplification*) y fragmentación en disco.

6. Conclusión Técnica Fundamentada (Consigna 7)

Decisión Arquitectónica y Criterios de Selección (Límite ≤ 200 palabras):

Para el sistema Food Store, la estrategia técnica óptima no reside en una migración excluyente, sino en una **arquitectura híbrida (políglota)** fundamentada en dos criterios técnicos esenciales:

1. **Integridad transaccional y validación estricta en el checkout (PostgreSQL):** La fase de compra exige reserva concurrente de stock (FOR UPDATE), control de crédito del cliente y constraints CHECK que verifiquen que la sumatoria de las líneas coincida de forma exacta con el total liquidado. PostgreSQL provee consistencia ACID estricta y aislamiento serializable insustituibles en el núcleo operativo para evitar condiciones de carrera financieras.

2. **Localidad de datos y throughput masivo de lectura en pedidos cerrados (MongoDB):** Una vez que el pedido pasa a estado confirmado o en entrega, su acceso muta hacia lecturas concurrentes intensivas (seguimiento en tiempo real por repartidores, visualización en apps de clientes y facturación). El modelo documental embebido elimina la contención de bloqueos y la sobrecarga de JOINs en PostgreSQL, permitiendo escalar horizontalmente las lecturas y archivar históricos inmutables sin degradar el rendimiento relacional.

Por ende, se recomienda mantener PostgreSQL como *Single Source of Truth* transaccional y sincronizar asíncronamente los pedidos confirmados hacia MongoDB como vista materializada de alta disponibilidad. *(198 palabras)*

7. Inventario de Entregables Adjuntos y Guía de Reproducción

Se adjuntan al presente informe los archivos ejecutables y fuentes que respaldan íntegramente los ensayos realizados:

Archivo Entregable	Descripción Técnica y Contenido	Comando de Ejecución Sugerido
consultas_mongodb.js	Script ejecutable para mongosh. Contiene la selección de base, creación de colección pedidos, migración con insertMany(), consultas 4.a, 4.b, 4.c, 4.d y agregación de control financiero.	`mongosh --file consultas_mongodb.js`
consultas_postgresql.sql	Script relacional para PostgreSQL 17. Contiene la inspección de cardinalidades, consultas equivalentes a 4.b y 4.c con JOINs, filtros de alcance y validación de totales.	`psql -U postgres -d foodstore -f consultas_postgresql.sql`
migrar_datos.py	Pipeline automatizado en Python que extrae pedidos de PostgreSQL y los inserta en MongoDB usando psycopg2 y pymongo.	`python migrar_datos.py`
BITACORA.md	Bitácora completa del uso de Inteligencia Artificial con formato de 3 columnas (Prompt, Respuesta, Corrección Humana) y contrastes técnicos.	Visualizable en Markdown / GitHub
README.md	Guía operativa de reproducción rápida, tablas de verificación y documentación del repositorio de trabajo.	Visualizable en Markdown / GitHub

Alumno: Maximiliano Nardez | Tecnicatura Universitaria en Programación | Bases de Datos II — Unidad 7